

学以致用

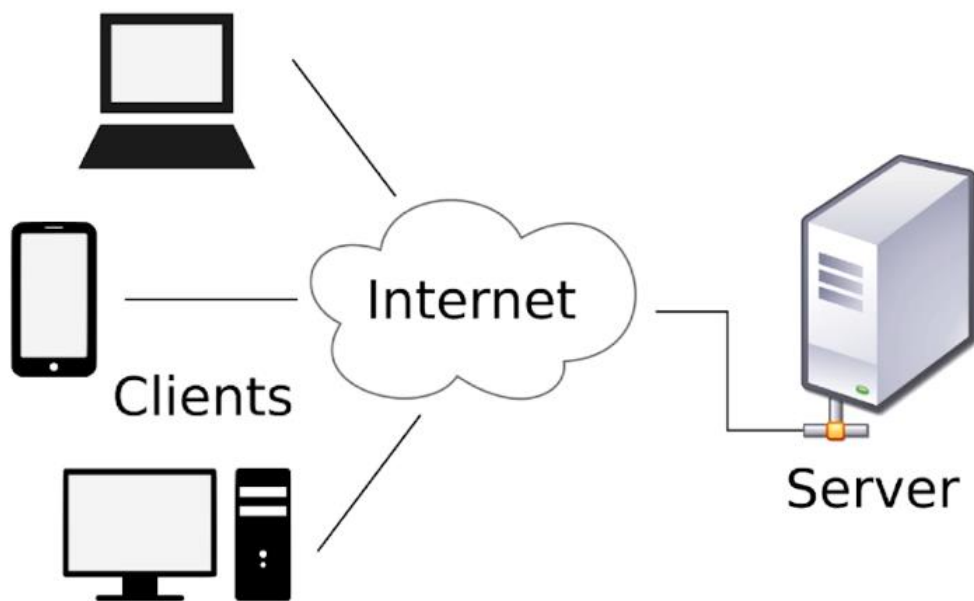


简易网络聊天室开发

北京理工大学计算机学院
金旭亮

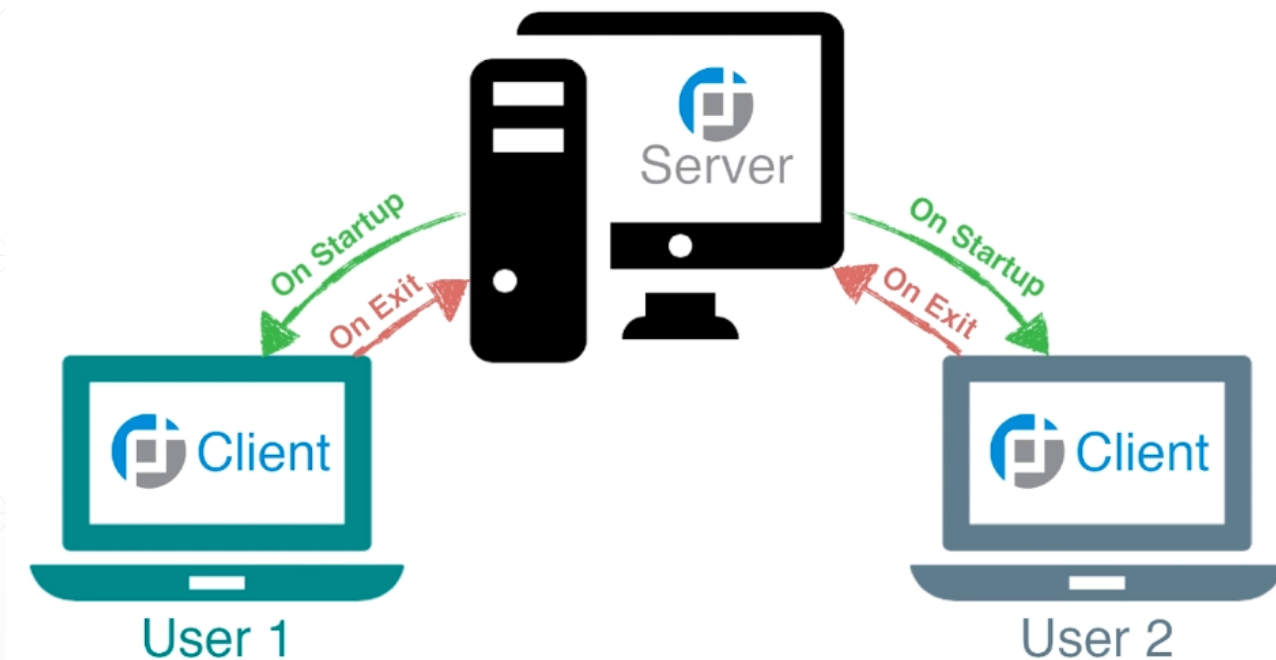
概述

在学习完了TCP的基础知识之后，让我们将学到的知识应用于实际，开发一个网络聊天室。



网络聊天室的主要功能就是让一群人在线上实时交谈，有一台服务器实现消息中转服务。一个人发出的消息，能被其他在线用户同时看到，反过来，他也能看到其他人发的消息。

网络聊天室基本工作原理



服务器监听客户端连接请求，通知聊天室在线用户，某某进入或离开；接收客户端发来的消息，解析它的信息（谁发的），然后将它显示到聊天室界面上。

运行截图

```
Windows PowerShell
PS D:\NetChatRoom\server> java ChatServer
启动服务器, 监听端口: 8888...
客户端[1945]已连接到服务器
客户端[2721]已连接到服务器
客户端[2721]: 大家好!
客户端[1945]: 2721, 你好!
客户端[2721]: 1945, 很高兴遇见你!
客户端[1945]: 2721, 我也是!
```

Server端

```
Windows PowerShell
PS D:\NetChatRoom\client> java ChatClient
客户端[2721]: 大家好!
2721, 你好!
客户端[2721]: 1945, 很高兴遇见你!
2721, 我也是!
quit
关闭socket
PS D:\NetChatRoom\client> |
```

客户端输入quit,
代表“下线”。

Client1

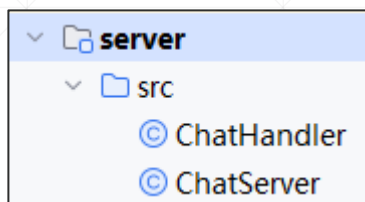
```
Windows PowerShell
PS D:\NetChatRoom\Client> java ChatClient
大家好!
客户端[1945]: 2721, 你好!
1945, 很高兴遇见你!
客户端[1945]: 2721, 我也是!
客户端[1945]: quit
```

在线的其他客户端,
会收到“下线”通知。

Client2

Server端代码解析

Server端的代码



Server端只有两个类，其中，ChatServer负责启动聊天室，并提供聊天室相关的公用功能（比如转发消息，用户上线下线等）

ChatHandler类用于响应客户端的连接请求，它会调用ChatServer所封装的功能，完成消息读取和转发消息等功能。

当有一个连接进来时，ChatServer会new一个ChatHandler对象，并且将自身引用传给它，以便让ChatHandler中的代码可以调用聊天室相关的功能。注意，ChatHandler实现了Runnable接口，它的代码，是在一个独立的线程中运行的。

```
public class ChatServer {  
    //监听端口  
    2 usages  
    private int DEFAULT_PORT = 8888;  
    //客户端退出命令（即“下线”）  
    1 usage  
    private final String QUIT = "quit";  
    //用于监听客户端连接的端口  
    4 usages  
    private ServerSocket serverSocket;  
    //用于保存在线用户信息  
    7 usages  
    private Map<Integer, Writer> connectedClients;
```

在线用户管理

//有客户端连接进来之后，将其加入到在线用户清单中，同时在控制台窗口输出一条信息

```
1 usage
public synchronized void addClient(Socket socket) throws IOException {
    if (socket != null) {
        int port = socket.getPort();
        BufferedWriter writer = new BufferedWriter(
            new OutputStreamWriter(socket.getOutputStream())
        );
        connectedClients.put(port, writer);
        System.out.println("客户端[" + port + "]已连接到服务器");
    }
}
```

用户离开聊天室时，只需将其记录从Map中移除就行了。

由于本示例仅在单机上运行，因此，使用端口号作为key，将在线用户信息保存到HashMap中。

由于Server端主要就是起一个转发作用，因此，它直接由客户端Socket得到一个BufferedWriter，保存到HashMap中，以简化代码。

//用户下线

```
1 usage
public synchronized void removeClient(Socket socket) throws IOException {
    if (socket != null) {
        int port = socket.getPort();
        if (connectedClients.containsKey(port)) {
            connectedClients.get(port).close();
        }
        connectedClients.remove(port);
        System.out.println("客户端[" + port + "]已断开连接");
    }
}
```

ChatServer实现的功能

消息的转发

```
//一个用户发来消息, Server转发消息给其他的在线用户
1 usage
public synchronized void forwardMessage(Socket socket, String fwdMsg)
    throws IOException {
    for (Integer id : connectedClients.keySet()) {
        if (!id.equals(socket.getPort())) {
            Writer writer = connectedClients.get(id);
            writer.write(fwdMsg);
            writer.flush();
        }
    }
}
```

两个辅助函数

```
//判断用户发来的消息是不是“下线”请求
1 usage
public boolean readyToQuit(String msg) {
    return QUIT.equals(msg);
}

//关闭聊天服务
1 usage
public synchronized void close() {
    if (serverSocket != null) {
        try {
            serverSocket.close();
            System.out.println("关闭serverSocket");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```


ChatServer启动聊天服务

当有客户端连接请求进来时，创建一个新线程处理这个请求.....

```
public static void main(String[] args) {  
    ChatServer server = new ChatServer();  
    //启动聊天服务  
    server.start();  
}
```

```
//启动聊天服务  
1 usage  
public void start() {  
    try {  
        // 绑定监听端口  
        serverSocket = new ServerSocket(DEFAULT_PORT);  
        System.out.println("启动服务器, 监听端口: " + DEFAULT_PORT + "...");  
        while (true) {  
            // 等待客户端连接  
            Socket socket = serverSocket.accept();  
            // 创建ChatHandler线程  
            new Thread(new ChatHandler(server: this, socket)).start();  
        }  
    } catch (IOException e) {  
        e.printStackTrace();  
    } finally {  
        close();  
    }  
}
```

响应客户端连接请求

//负责处理客户端连接请求

1 usage

```
public class ChatHandler implements Runnable {  
    5 usages  
    private ChatServer server;  
    6 usages  
    private Socket socket;  
    1 usage  
    public ChatHandler(ChatServer server, Socket socket) {  
        this.server = server;  
        this.socket = socket;  
    }  
}
```

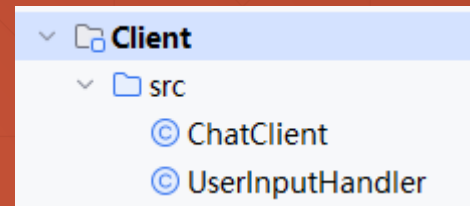
@Override

```
public void run() {  
    try {  
        // 存储新上线用户  
        server.addClient(socket);  
        // 读取用户发送的消息  
        BufferedReader reader = new BufferedReader(  
            new InputStreamReader(socket.getInputStream())  
        );  
        String msg = null;  
        while ((msg = reader.readLine()) != null) {  
            String fwdMsg = "客户端[" + socket.getPort() + "]: " + msg + "\n";  
            System.out.print(fwdMsg);  
            // 将消息转发给聊天室里在线的其他用户  
            server.forwardMessage(socket, fwdMsg);  
            // 检查用户是否准备退出  
            if (server.readyToQuit(msg)) {  
                break;  
            }  
        }  
    }  
}
```

ChatHandler的基本职责，就是接收客户端发来的消息，将其转发给其他用户，然后，检查一下是不是"quit"，如果是，则退出线程，并将其从“在线用户”清单中移除。

```
} catch (IOException e) {  
    e.printStackTrace();  
} finally {  
    try {  
        server.removeClient(socket);  
    } catch (IOException e) {  
        e.printStackTrace();  
    }  
}
```

Client端代码解析



ChatClient封装的相关信息与消息的发送与接收

```
public class ChatClient {  
    //服务器IP地址  
    1 usage  
    private final String DEFAULT_SERVER_HOST = "127.0.0.1";  
    //服务器监听端口  
    1 usage  
    private final int DEFAULT_SERVER_PORT = 8888;  
    //“下线”命令  
    1 usage  
    private final String QUIT = "quit";  
    5 usages  
    private Socket socket;  
    //接收消息  
    2 usages  
    private BufferedReader reader;  
    //发送消息  
    5 usages  
    private BufferedWriter writer;
```

```
// 发送消息给服务器  
1 usage  
public void send(String msg) throws IOException {  
    if (!socket.isOutputShutdown()) {  
        writer.write(str: msg + "\n");  
        writer.flush();  
    }  
}
```

```
// 从服务器接收消息  
1 usage  
public String receive() throws IOException {  
    String msg = null;  
    if (!socket.isInputShutdown()) {  
        msg = reader.readLine();  
    }  
    return msg;  
}
```

ChatClient实现下线操作

// 检查用户是否准备退出

1 usage

```
public boolean readyToQuit(String msg) {  
    return QUIT.equals(msg);  
}
```

// 关闭Socket并退出

1 usage

```
public void close() {  
    if (writer != null) {  
        try {  
            System.out.println("关闭socket");  
            writer.close();  
        } catch (IOException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

客户端执行流程

```
public void start() {  
    try {  
        // 创建socket  
        socket = new Socket(DEFAULT_SERVER_HOST, DEFAULT_SERVER_PORT);  
        // 创建IO流  
        reader = new BufferedReader(  
            new InputStreamReader(socket.getInputStream())  
        );  
        writer = new BufferedWriter(  
            new OutputStreamWriter(socket.getOutputStream())  
        );  
    }  
}
```

客户端的启动:

```
public static void main(String[] args) {  
    ChatClient chatClient = new ChatClient();  
    chatClient.start();  
}
```

在另外一个线程中接收用户键盘输入.....

```
// 处理用户的输入  
new Thread(new UserInputHandler(chatClient: this)).start();  
// 读取服务器转发的消息  
String msg = null;  
while ((msg = receive()) != null) {  
    System.out.println(msg);  
}  
} catch (IOException e) {  
    e.printStackTrace();  
} finally {  
    close();  
}  
}
```

将收到的信息显示在控制台窗口中

处理用户输入



//从键盘接收用户输入，然后将消息发送给Server

```
1 usage
public class UserInputHandler implements Runnable {
    3 usages
    private ChatClient chatClient;
    1 usage
    public UserInputHandler(ChatClient chatClient) {
        this.chatClient = chatClient;
    }
}
```

@Override

```
public void run() {
    try {
        // 等待用户输入消息
        BufferedReader consoleReader =
            new BufferedReader(new InputStreamReader(System.in));
        while (true) {
            String input = consoleReader.readLine();
            // 向服务器发送消息
            chatClient.send(input);
            // 检查用户是否准备退出
            if (chatClient.readyToQuit(input)) {
                break;
            }
        }
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```



问题、改进与重构

改进与重构-1



Server端应该保存客户端的IP、Port、称呼、头像等信息，所以，应该定义一个专门的类。



消息的发送与接收，应该统一编码，代码中应该显式设置为UTF-8。



Server端应该使用线程池，而不是为每一个客户端连接请求都创建一个新的线程。



应该要有一个心跳检测功能，以便确保客户端“真”的在线，长时间不发送消息的客户，Server端可将其“踢”出去，这个功能，可以使用UDP实现。

改进与重构-2



服务端和客户端都是两个类，这两个类之间的代码紧耦合，不易于维护与扩展。



当前仅仅是在控制台中发送与接收消息，应该给它套上一个GUI界面（Swing或JavaFX）。



当前只支持纯文本消息，可以扩充功能，支持二进制消息，实现文件的群发功能。